

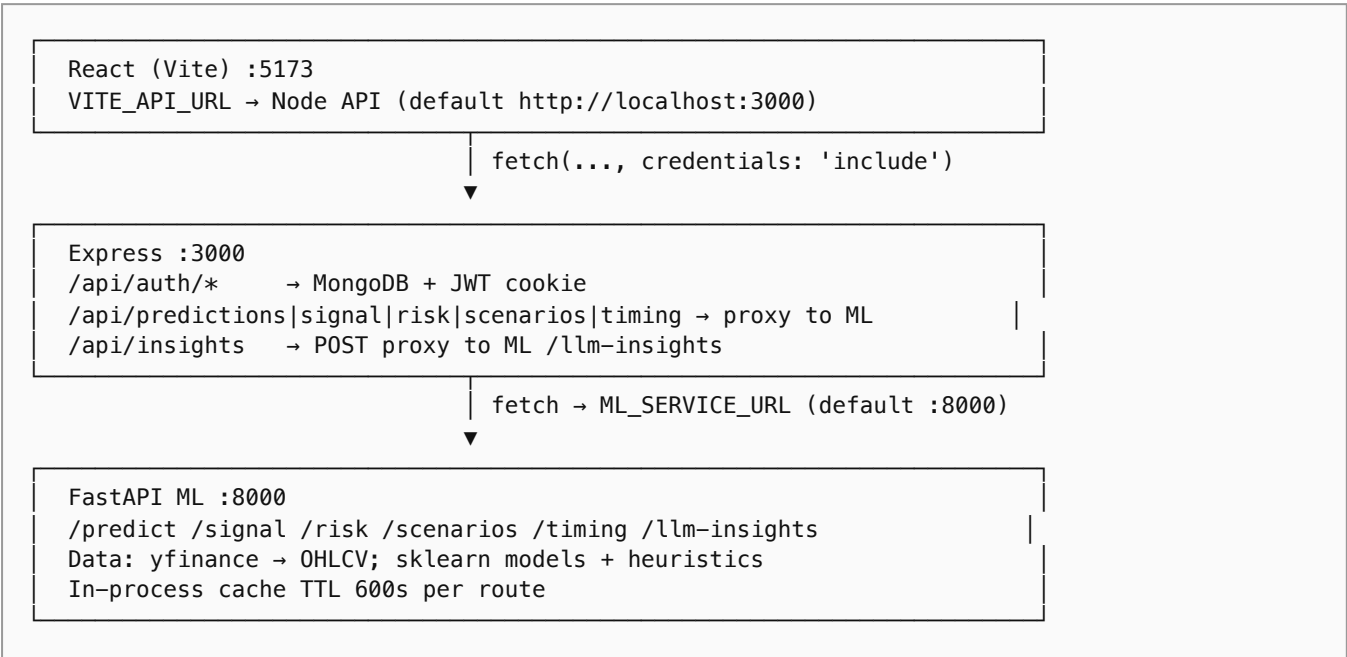
NeonTrade AI

System Architecture & ML Integration

System Architecture Audit · Technical documentation

NeonTrade AI — System Architecture Audit

High-level picture



Axios: Not used in `client/src`; all browser calls use `fetch`. The Node server also uses `fetch` to call FastAPI (not axios).

Frontend (React)

Routing (`App.tsx`)

Area	Routes
Public	<code>/</code> , <code>/signup</code> , <code>/login</code>
Dashboard	<code>/dashboard</code> → redirect to <code>/dashboard/profile</code>
Overview	<code>/overview</code> → <code>/overview/short-term</code> ; also <code>/overview/intraday</code> , <code>/overview/long-term</code>
Predictions	<code>/predictions</code> → <code>/predictions/short-term</code> ; also intraday & long-term
Compare	<code>/compare</code>
Guard	<code>ProtectedRoute</code> + <code>ProfileCompletionGuard</code> ($\geq 80\%$ profile) for Overview / Predictions / Compare

Main pages (what you asked)

Overview (Intraday / Short-term / Long-term)

Implemented by `OverviewPage` with `type: 'intraday' | 'short-term' | 'long-term'`.

- **Data:** On mount, **parallel** `fetchSignal` for a fixed list of tickers per horizon (`horizonToTopPicksTickers`). Successful responses replace **Top AI Picks** with live signal fields mapped into `StockRecommendationCard`s.
- **Charts:** **Bar** “confidence comparison” is **derived from live top picks** when signals succeed (`buildSafeChartData`). The **line** chart still uses **static** `dataset.chartDataPredictedVsCurrent` from `overview/overviewDatasets.ts`.
- **Mock/static:** Market snapshot cards, sector breakdown, initial/fallback top picks, default AI insight text, and the **predicted vs current line series** come from `getOverviewDataset()` (built on `overviewDatasets.ts` + types from `overviewMockData.ts`).
- **LLM:** After picks update, `fetchInsights('overview', ...)` → `POST /api/insights`; success updates `AIInsightCard` text.

Predictions (Intraday / Short-term / Long-term)

Three pages share the same pattern: `PredictionTemplate` + ticker `<select>`.

- **Data:** `Promise.all` of `fetchPredictions`, `fetchRisk`, `fetchScenarios`, `fetchTiming` for the selected ticker and that page's horizon.
- **UI pipeline:** Price path → `PriceForecastChart`; scenarios → `ScenarioCard`; timing → `TimingHeatmap`; derived volatility vs SPY comparison in `PredictionComparisonChart` / `VolatilityForecast` uses **template logic** mixing API risk + price bands; `fetchInsights('prediction', ...)` can replace `AIReasoningPanel` text.
- `predictionsMockData.ts`: Used mainly for **TypeScript types** (`PriceTrajectoryPoint`, etc.). Static arrays at the bottom are **legacy demo data**, not wired in these three pages.

Compare

- **Asset list:** `compareMockData.assetPool` only (searchable tickers).
- **After “Run Comparison”:** `fetchSignal`, `fetchPredictions`, `fetchTiming` per selected asset (2–4). Charts/matrix/scatter/timeline are filled from **live responses**. Horizon mapping: **Short Term & Medium Term** → `short-term`; **Long Term** → `long-term` API horizon.
- **LLM:** `fetchInsights('compare', ...)` can overwrite `AIVerdictPanel` when `verdict` returns.
- **Unused mock exports:** `comparisonSummaries`, `trajectoryComparisonData`, etc. in `compareMockData.ts` are **not** used when the user runs a comparison (only types + `assetPool`).

Dashboard / Profile (`DashboardProfilePage`)

- **No trading APIs.** Profile load/save goes through `profileApi.ts`: simulated delay + `localStorage` keyed by user id. Stats and “linked brokers” are **placeholder / mock**. Auth user comes from `/api/auth/me` via `AuthContext`.

How data reaches charts / cards

- **Overview:** Cards ← `topPicks` state; bar chart ← `dynamicBarData` from picks; line chart ← static dataset; insight card ← `insightText`.
- **Predictions:** Charts consume `priceTrajectoryData` from `/api/predictions`, risk/scenarios/timing from their routes; template **derives** scenarios UI (with API scenarios preferred), timing slots, volatility series, and SPY comparison row.
- **Compare:** Summary strip, multi-line chart, matrix, scatter, timeline built in `ComparePage` from merged API maps.

API surface used by the client

Service file	HTTP	Backend path
<code>predictionsApi.ts</code>	GET	<code>/api/predictions?ticker&horizon</code>
<code>signalApi.ts</code>	GET	<code>/api/signal?ticker&horizon</code>
<code>riskApi.ts</code>	GET	<code>/api/risk?ticker&horizon</code>
<code>scenarioApi.ts</code>	GET	<code>/api/scenarios?ticker&horizon</code>
<code>timingApi.ts</code>	GET	<code>/api/timing?ticker&horizon</code>
<code>insightsApi.ts</code>	POST	<code>/api/insights</code> body { type, data }
<code>AuthContext</code> / login pages	GET/POST	<code>/api/auth/me</code> , signup, login, logout

Base URL: `API_BASE_URL` from `lib/constants.ts` (`VITE_API_URL` or `http://localhost:3000`).

Backend (Express)

Mounted routes (`server/src/index.ts`)

- GET `/api/health`
- `/api/auth` → signup, login, logout, me (real Mongo + JWT cookie)
- `/api/predictions` → ML GET `/predict`
- `/api/signal` → ML GET `/signal` (with 404 → derive from `/predict` fallback in Node)
- `/api/risk` → GET `/risk`
- `/api/scenarios` → GET `/scenarios`
- `/api/timing` → GET `/timing`
- `/api/insights` → POST `/llm-insights`

ML base URL: `process.env.ML_SERVICE_URL` ?? `'http://127.0.0.1:8000'`.

Validation: Each ML-proxy route (except insights) restricts **tickers** and **horizons** to the same allowlists as in the route files (aligned with Python `SUPPORTED_TICKERS` / horizons).

Caching (Node, in-memory Map):

Route	TTL
Signal	2 min
Risk	5 min
Scenarios	10 min
Timing	3 min
Insights	5 min
Predictions	none in Node (ML still caches)

Incomplete / stubbed

- POST/PUT/DELETE on `/api/predictions` → **501** “not implemented”.

Error behavior

- ML unreachable or network error → **502** with message + details.
- ML returns non-OK → forward status and ML JSON as `details`.

ML Service (FastAPI)

Endpoints (`ml-service/app.py`)

HTTP	Path	Role
GET	<code>/health</code>	Health
GET	<code>/predict</code>	Price series + bands (<code>model.generate_predictions</code>)
GET	<code>/risk</code>	Risk scores (<code>risk_model.generate_risk_assessment</code>)
GET	<code>/signal</code>	Trading signal (<code>signal_model.generate_signal</code> ; uses price + optional risk)
GET	<code>/scenarios</code>	Monte-Carlo-style scenarios (<code>scenario_model.generate_scenarios</code>)
GET	<code>/timing</code>	Per-slot timing (<code>timing_model.generate_timing</code>)
POST	<code>/llm-insights</code>	Narrative (<code>llm_service.generate_llm_insight</code>)
GET	<code>/meta/supported-tickers</code>	List from <code>utils.SUPPORTED_TICKERS</code>

Caching: In-memory dicts, **TTL 600s** per (`ticker` , `horizon`) (and LLM by request key).

Data source: `data_loader.fetch_ohlcv_data` uses `yfinance.download` with horizon-specific interval/period from `HORIZON_CONFIG` in `utils.py`.

Models (implemented vs “synthetic training”)

Module	What runs
<code>model.py</code>	RandomForestRegressor on real OHLCV features; iterative multi-step forecast.
<code>signal_model.py</code>	RandomForestClassifier trained on synthetic feature/label data; inference uses real features from prediction path (+ risk when present).
<code>risk_model.py</code>	RandomForestRegressor trained on synthetic X; real volatility, drawdown, uncertainty, beta-style inputs at inference.
<code>scenario_model.py</code>	Monte Carlo from drift/vol derived from predictions + risk; no separate sklearn file on disk.
<code>timing_model.py</code>	RandomForestClassifier on synthetic training; real RSI/MACD/volume-style features from history + predictions.
<code>llm_service.py</code>	OpenAI API if <code>OPENAI_API_KEY</code> set; else exception → template fallback (no LLM).

So: **price path is the most “data-grounded”**; several classifiers/regressors are **real sklearn** but **not trained on historical labels**—they use simulated training sets.

Data Flow

Overview (e.g. Short-term)

```
OverviewPage
  → fetchSignal(NVDA, 'short-term') × N tickers
    → GET http://localhost:3000/api/signal?...
      → GET http://127.0.0.1:8000/signal?...
        → generate_predictions (yfinance + RF)
        → optional risk pipeline
        → generate_signal
  → UI: Top picks + dynamic confidence bars
  → fetchInsights('overview', ...)
    → POST /api/insights → POST /llm-insights → OpenAI or fallback text
```

Static pieces **do not** hit the ML API: snapshot cards, sectors, line chart series, fallback picks.

Predictions (e.g. Short-term)

```
PredictionsShortTermPage
  → Promise.all(
    GET /api/predictions → ML /predict
    GET /api/risk        → ML /risk
    GET /api/scenarios  → ML /scenarios
    GET /api/timing     → ML /timing
  )
  → PredictionTemplate maps results into charts + derived panels
  → fetchInsights('prediction', ...) → optional LLM reasoning text
```

Compare

```
ComparePage “Run Comparison”
  → Promise.all per ticker: signal, predictions, timing
  → Client merges into trajectory table, matrix, scatter, timeline
  → Initial verdict string from live summaries; then fetchInsights('compare') may replace it
```

Note: Compare does **not** call `/api/risk` or `/api/scenarios` directly; risk-like numbers on the matrix come from **signal fields** (`risk_score` , `drawdown_risk` , `features`) when the ML signal payload includes them (signal merges risk when available).

Mock vs Real Data

Page	Uses mock / static content?	API connected?	ML connected (via Node)?
Landing / Auth	UI copy only	Auth → real Mongo API	No
Dashboard / Profile	Profile + stats local mock	Auth only	No
Overview	Snapshots, sectors, line chart series, fallback picks, default insight	Yes (<code>/api/signal</code> , <code>/api/insights</code>)	Yes when backend+ML up
Predictions (all horizons)	Types from <code>predictionsMockData</code> ; optional derived SPY row / template text	Yes (predict, risk, scenarios, timing, insights)	Yes
Compare	Only <code>assetPool</code> labels; demo arrays unused at runtime	Yes after Run	Yes

Issues / Gaps (current)

1. **Split truth:** Overview **line** chart and several headline metrics are **static** while picks are **live**—easy to **look inconsistent** (e.g. “Avg Confidence” card vs live bars).

2. **QQQ in intraday overview dataset** (`overviewDatasets.ts`) is **not** in ML/server allowlists—live calls only use the configured ticker list; mismatch only matters if you ever wire that ticker to the API.
3. **Signal route 404 fallback** in Node targets an old “ `/signal` missing” case; FastAPI **does** expose `/signal` , so this path is mostly **dead** unless the ML service changes.
4. **POST `/api/predictions` returns 501**—no persistence layer for predictions in the app.
5. **LLM insights** depend on `OPENAI_API_KEY` on the **ML** host; otherwise users always see **deterministic fallback** copy.
6. **End-to-end failure:** If Node or ML is down, Overview **falls back** to dataset picks; Predictions **show error** and empty charts.
7. **Training vs narrative:** Several models use **synthetic training data**; for interviews, be precise: “**real market features at inference, synthetic labels for classifier training.**”
8. **No axios / no Vite proxy:** Client must point `VITE_API_URL` at the API; no automatic same-origin proxy in `vite.config.ts` .

Final Architecture Summary

React (Vite)

- cookies + JWT for session
- fetch → Express on `:3000` (except profile mock)

Express

- Auth + MongoDB
- Proxies + validates → FastAPI
- In-memory caches on `signal/risk/scenarios/timing/insights`

FastAPI

- `yfinance` (Yahoo Finance) for OHLCV
- `sklearn` pipelines + scenario MC + optional OpenAI
- 600s in-memory cache per key

Failure points

- Yahoo empty/failure → ML 400/500 → Node forwards or 502
- ML down → Node 502 → UI errors or Overview fallback
- OpenAI missing → templated insight text only

Optional improvements (architecture only)

1. **Unify Overview:** Drive **all** overview visuals from the same API responses (or one aggregated `/api/overview` that calls ML once per horizon) so snapshots and charts match **Top Picks**.
2. **Replace mock first:** `overviewDatasets` **static line chart + snapshot cards**—highest user-visible inconsistency; then **profile** if you need a real product story.
3. **Stabilize API flow:** Single **BFF-style** endpoint per page that returns a **versioned schema**; add **timeouts + partial success** (Overview already uses `allSettled` for picks—extend that pattern).
4. **ML honesty:** Persist or document **model cards** (training data, limitations); consider training `signal/risk/timing` on **historical labels** if you claim “ML predictions” in interviews.

5. **Operational:** Shared **Redis** for cache if you scale beyond one Node/ML process; health checks that verify **Mongo + ML + yfinance** reachability.

This should give you a **single map** of NeonTrade: **what is mock, what is live, where data flows, and where interviews or debugging should focus** (Express proxy → FastAPI → yfinance + sklearn + optional OpenAI).